

Future Thoughts: On Facetwork's Positioning in the AI-Agent Era — A Dissenting Companion

1. Where the dominant framing is right
2. Where the dominant framing overshoots
 - 2.1 "Target existing engines" undersells FFL's semantics
 - 2.2 The human-DSL versus AI-IR dichotomy is too clean
3. Restatement
4. What this implies for the roadmap
5. Working evidence: the structured-diagnostics feedback loop

Future Thoughts: On Facetwork's Positioning in the AI-Agent Era — A Dissenting Companion

A short companion to `future-thoughts-ai-native.md` and `ai-authorship.md`.

This note records one reader's partial agreement and partial pushback against the now-common framing that a new workflow system — including Facetwork — has lost most of its standalone value because AI agents can generate workflows for mature engines (Temporal, Step Functions, Prefect, LangGraph, Dapr Workflows). The dominant reframing says: *don't compete on syntax, compete on validation, policy, repair, provenance, and safe execution of AI-generated plans — possibly by targeting existing runtimes*. This note agrees with most of that, and dissents on two points.

1. Where the dominant framing is right

The core claim holds up. If Facetwork's pitch is "a nicer DSL so humans can author workflows more easily," the pitch is weaker in 2026 than it was in 2018. Three reasons:

- **AI agents don't reward elegant syntax.** An agent generating a Temporal workflow, a Step Functions state machine, or an Airflow DAG does not particularly benefit from a smaller grammar. Given documentation and examples, agents produce correct definitions in verbose, imperative target languages just as readily as in a compact DSL.
- **Mature engines have runtime moats that are expensive to replicate.** Durability, retries, scaling, observability, connectors, and — crucially — the operational trust accumulated over years of production usage. A new runtime cannot shortcut that.
- **The binding constraint has moved.** The hard problem is no longer "how do I express this workflow?" It is "how do I verify, bound, explain, and repair a workflow that was generated by a system I did not fully supervise?" That is a validation and control problem, not a syntax problem.

So the reframing is sound. The load-bearing pieces of Facetwork, in this light, are not the surface syntax of FFL but the machinery around it: the typed schemas, the compiler-level validation, the step recovery actions, the repair tooling, the execution traces. Those are AI-native control-plane features, whether or not they were originally framed that way.

2. Where the dominant framing overshoots

Two pushbacks.

2.1 "Target existing engines" undersells FFL's semantics

The natural next move from the reframing is: *let Facetwork be a frontend — an AI-native IR and validator — that compiles down to Temporal or Step Functions*. This is tempting because it inherits the runtime moat. But it flattens the thing that may actually matter.

FFL's composition model is not just syntactic sugar over Temporal's imperative workflow functions or Step Functions' state machines. Mixins, implicit facets, typed schemas, `andThen` `foreach`, statement-level `andThen`, `catch when`, prompt blocks, script blocks — these form a different *algebra*. Workflows compose by declaration, not by control flow. Effects and data dependencies are explicit in the structure, not buried in procedural code.

Why does that matter for AI-generated workflows? Because **compositional, declarative structure is easier to verify than imperative structure**. A validator can reason about a facet graph — its dependencies, its typed inputs and outputs, its declared effects — far more tractably than it can reason about an arbitrary Python function that happens to call `workflow.execute_activity()`. A smaller, more regular surface area shrinks the set of things a validator, a repair tool, or a human reviewer has to consider.

Compiling FFL to Temporal would erase that advantage. The target runtime's expressiveness leaks back into the IR, and the IR becomes "whatever Temporal can do," which is "arbitrary code." The control-layer framing only works if the IR is *more constrained* than the runtime it ultimately executes on. Facetwork's semantic restrictions are a feature, not an accident to be hidden behind a compiler.

So: yes to AI-native IR. No to reducing FFL to a pretty frontend for somebody else's runtime. The semantics are part of the product.

2.2 The human-DSL versus AI-IR dichotomy is too clean

The reframing tends to sort workflow systems into two camps: human-facing DSLs (declining in value) and machine-facing IRs (rising in value). Facetwork, in this sorting, is pushed to pick a side and told to pick the IR side.

But the high-value artifact in an AI-agent workflow may be precisely the thing that is *both*. When an agent proposes a workflow, a human reviewer needs to read it to approve it. A pure IR — JSON, protobuf, a Merkle DAG of hashes — loses the review affordance. Reviewers will not read thousands of lines of serialized graph nodes and catch subtle misbehavior. They need a projection that is dense, readable, and structurally honest about what will execute.

A well-designed human-readable DSL, if its semantics are tight, *is* that projection. FFL is in that middle zone: readable enough that a reviewer can see the shape of what was generated, structured enough that a compiler can validate it. A companion note (`future-thoughts-ai-native.md` §1) captures this as "the language is really two things: a canonical IR (what executes) and a projection layer (what a human sees when verifying)." That is exactly the point. FFL's surface syntax is not a legacy concession to human authors; it is the review interface for AI-authored plans.

The risk of the pure-IR framing is that it dismisses the projection as nostalgia. It is not. It is the seam where human oversight actually happens.

3. Restatement

Synthesizing agreement and dissent:

- **Agreed:** Facetwork's strongest modern positioning is as an AI-native workflow specification and control layer, centered on validation, policy, repair, and provenance.
- **Dissented:** That positioning does not imply abandoning FFL's semantics in favor of existing runtimes, nor collapsing the human-readable surface into a pure machine IR. FFL's compositional algebra is part of what makes validation tractable, and its readable surface is part of what makes human oversight possible.

The sharper version of the pitch, then, is not:

Facetwork is an AI-native workflow specification and control layer that helps agents generate, validate, explain, repair, and safely execute workflows.

But rather:

Facetwork is an AI-native workflow system whose semantic restrictions make AI-generated plans tractable to validate and repair, and whose readable surface makes them tractable to review.

The claim rests on two structural properties — restricted semantics and a readable projection — that reinforce each other. Drop either one and the system loses what distinguishes it from both the mature runtimes on one side and the pure-IR proposals on the other.

4. What this implies for the roadmap

Concretely, if this framing is right:

- **Keep the semantic restrictions.** Resist feature requests that would make FFL arbitrarily expressive. Every restriction is a validator's friend.
- **Invest in the projection layer as a first-class review interface.** Execution traces, diff-against-prior-run views, natural-language paraphrases of generated plans, and effect summaries are not UI polish — they are the oversight mechanism.
- **Treat the IR and the surface as a pair.** If a proposed feature is readable but not validatable, or validatable but not readable, it probably does not belong.
- **Be skeptical of "just compile to Temporal."** That path is available, but taking it would spend the semantic advantage to buy a runtime moat Facetwork does not necessarily need at its stage.

None of this contradicts the dominant reframing. It sharpens it: the AI-native control layer works because of FFL's semantics and because of FFL's surface, not in spite of them.

5. Working evidence: the structured-diagnostics feedback loop

The positioning above is testable. If FFL's restricted semantics make validation tractable for AI authors, then we should be able to give an agent the same diagnostic-driven repair loop it already enjoys for Python or TypeScript: error → linked docs → wrong/right examples → fix → re-check. The recent validator and MCP changes are a deliberate test of that claim.

Concretely, the validator now emits structured `ValidationError` records carrying not just a message and a source location but a stable `rule_id`, a `severity`, a `docs_uri`, and an optional `suggested_fix`. Forty-six call sites across the validator have been threaded with explicit `rule_ids` spanning nine categories — reference resolution, `when` exhaustiveness, type checking, yield targets, prompt and script blocks, schema and implicit-facet placement, and structural restrictions on where workflows and schemas may appear. Every emitted `rule_id` has a paired markdown file at `docs/reference/rules/{rule_id}.md` containing a short rationale, a wrong example, a right example, and the underlying constraint it enforces. The set is currently exact: 41 emitted `rule_ids`, 41 documented `rule_ids`, with a script that diffs the two on every change.

The MCP layer exposes this without flattening it. A call to `fw_validate` returns the full diagnostic structure — `rule_id`, `severity`, `location`, `docs_uri`, `suggested_fix`, plus warnings — rather than a single English error string. The companion resource `afl://docs/rules/{rule_id}` returns the rule's documentation, so an agent that hits a diagnostic can fetch its own remediation context in one step. A second resource exposes a small set of canonical, validator-clean FFL examples, so the agent has a known-good starting point when the failing source is too tangled to repair in place.

Two things are worth noting about why this works at all, and they come straight from the dissent above.

First, the rule taxonomy is small and stable because the language is small and stable. The validator emits 41 distinct rules, not 410. That number is finite for the same reason FFL is restricted: the grammar does not let workflows or schemas float free of a namespace; references must take `step.field` form; `when` blocks must end in a default; loop variables are scoped; comparisons return Boolean and `&&` / `||` / `!` accept only Boolean operands. Each of those restrictions is an axis along which the validator can speak precisely. A more permissive language would not have a closed enumeration of failure modes — it would have a long tail of "here's a parser error, good luck." The compositional algebra in §2.1 is what makes the `rule_id` space tractable to enumerate and tractable to document.

Second, the suggested fixes and the docs uri are valuable to a *human reviewer* of agent-generated FFL, not only to the agent. A reviewer reading a generated workflow can click through to `afl://docs/rules/REF_INVALID_STEP_FORMAT` and see the same wrong/right framing the agent saw. The diagnostic surface is, in this sense, another instance of the projection layer argued for in §2.2: the same artifact serves machine repair and human verification. A pure-IR system would have to invent a separate review affordance for this; FFL inherits it because the surface and the semantics are paired.

This does not, on its own, prove the larger positioning. What it does demonstrate is that one of the load-bearing claims — *restricted semantics make AI-driven repair tractable in the same way restricted semantics make human review tractable* — is not aspirational. It is now the working contract between the compiler and any agent (Claude Code, an MCP client, a future planning agent) that touches FFL. The next claims to test are at the runtime layer: structured repair diagnostics for stuck workflows (the analogue at the execution layer of what the validator now does at the source layer), and effect annotations on event facets (so policy checks have something concrete to check against). Those follow the same template — a small, restricted vocabulary, paired with structured diagnostics and per-term docs — and they extend the same evidence.